

MÓDULO 4. Solución de problemas y técnicas de programación.

1. Etapas y metodologías en la solución de problemas.
 2. Elementos básicos de programación.
 3. Estructuras de control.
 4. Algoritmos y diagramas de flujo.
-

Como técnico en computación, una de las habilidades más importantes que debes desarrollar en tu trayectoria es la solución de problemas de programación. Este módulo aborda algunos enfoques, metodologías y elementos esenciales para ello.

1. Etapas y metodologías en la solución de problemas.

Antes que nada, es relevante saber cuál es la definición concreta de “**problema**” y entender cuál es la relación exacta entre tal concepto y el ámbito computacional o de programación.

Podríamos definir un problema como “**Una situación en la cual un individuo actúa con el propósito de alcanzar una meta utilizando para ello alguna estrategia en particular**”.

En tal caso la meta sería la solución. Tenemos, por otro lado, la **resolución de problemas**, que es el proceso cognoscitivo a través del cual se llega a la meta.

La resolución de problemas tiene diferentes etapas, podemos identificar, por ejemplo:

- El análisis y especificación del problema.
- La estrategia.
- La planeación de posibles soluciones.
- La ejecución de la solución.
- La comprobación de resultados.
- La corrección de errores (si es que existen).

En el caso de la computación, es usual que se nos presenten conflictos que pretendan ser resueltos a través de técnicas específicas de programación. Para ello, debemos tener en cuenta conceptos básicos como **algoritmos**, **diagramas de flujo** y **estructuras de control**.

Manejar tales conocimientos es parte importante de resolver problemas exitosamente.

Aunque muchas veces asumimos que entrar al editor y comenzar a escribir código es suficiente, en problemas complejos o de gran magnitud, eso no será de utilidad. Por ende, siempre se deben aplicar **metodologías** que nos permitan llegar a una solución correcta, eficiente y bien estructurada.

Las **metodologías de programación** son sistemas que nos ayudan a planificar y desarrollar software de manera eficaz. Al igual que la resolución de problemas, cuentan con una serie de pasos que debemos aplicar para llevarla a cabo correctamente.

Aunque existen diversos tipos de metodologías, comúnmente la mayoría de ellas siguen las mismas **etapas**:

- **Planteamiento.** Para definir qué problemas se deben resolver y qué es lo que se espera del resultado.
- **Análisis.** Para recopilar la información necesaria y pensar cómo será la solución.
- **Diseño.** Para definir estructuras, funciones y organización del código; además de plantear algoritmos y diagramas de flujo.
- **Programación.** Para construir la solución en un editor de código a través de algún lenguaje de programación.
- **Testeo.** Para comprobar que lo que se ha desarrollado funcione correctamente, cumpla con las especificaciones o expectativas de la solución y carezca de errores.

Existe una evidente similitud entre las técnicas de resolución de problemas y las metodologías de programación, precisamente porque ambos conceptos están estrechamente relacionados: **la correcta resolución de problemas depende de llevar a cabo una metodología eficaz.**

2. Elementos básicos de programación.

Para comenzar a desarrollar se deben conocer los elementos esenciales dentro de un programa que son de utilidad para prácticamente cualquier lenguaje de programación. Tener presente su funcionamiento será la clave para construir un software desde lo más básico hasta lo más complejo.

- **Variable y constante.**

Variable. En computación, una variable se puede definir como una **porción de memoria** física o virtual asignada para almacenar **un valor que puede cambiar**. Por ejemplo: un número, una letra o una cadena de caracteres.

Puede tener diferentes alcances dentro del código (local o global) y es esencial para construir expresiones lógicas.

En un programa, una variable siempre se debe declarar e inicializar para establecer su nombre, el tipo de dato que va a contener y opcionalmente su valor inicial.



Constante. Por otro lado, una constante también implica una porción de memoria ocupada por un valor, sin embargo, este **dato** es **predefinido y no puede cambiar** a lo largo de la ejecución del programa.

Aunque comparte ciertas características con las variables, como los diferentes alcances dentro del código y los tipos de datos que puede almacenar, su principal diferencia radica en que el valor que tiene asignado será fijo.

Para entender esta diferencia, se pueden plantear ambos conceptos en términos matemáticos utilizando expresiones algebraicas con las que ya estamos familiarizados. Por ejemplo: una ecuación ' $y + 1 = 3x - 2$ ', donde "y" y "x" son variables, mientras que los números 3 y 2 son constantes, pues su valor no cambia.

- **Tipos de datos.**

Un tipo de dato es el atributo que nos permite **organizar** y **clasificar** la información que se manipula a lo largo de un programa. Éste determina qué valores puede almacenar una variable, las operaciones que se podrán realizar con ella y el espacio de almacenamiento que ocupará. Existen diferentes tipos de datos, los más relevantes y que debes conocer por el momento son los siguientes:

Numéricos:

1. **int** (enteros).
2. **float** (con decimal, de 4 bits, precisión media).
3. **double** (con decimal, de 8 bits, precisión alta).

Alfanuméricos:

1. **char** (admite un solo carácter, por ejemplo 'A', '3', 's').

Lógicos:

1. **bool true** (de tipo booleano, equivale a 1).
2. **bool false** (de tipo booleano, equivale a 0).

(Los booleanos son datos que evalúan condiciones y ayudan a controlar el flujo de acciones dentro de un programa según su valor, el cual solo puede ser falso o verdadero).

Al momento de desarrollar software, debemos procurar tener presentes los distintos tipos de datos que nos permite un lenguaje de programación, pues estos son de suma importancia para plantear soluciones correctas y eficientes en términos de tiempo y memoria.



- **Operadores:**

Los operadores en programación se refieren a **símbolos** o **palabras reservadas** que representan alguna operación específica entre una o varias expresiones (operandos). Al igual que en los tipos de datos, se dividen en diferentes categorías según las acciones que realizan. Aunque para su estudio puedan parecer tediosos, en realidad son bastante sencillos de comprender y muchos de ellos inclusive los utilizamos en procesos lógicos y cognitivos cotidianos.

En programación estructurada con lenguaje C encontramos los operadores de asignación, los aritméticos, los lógicos y los de comparación. Para entender mejor el funcionamiento y la aplicación de cada uno te puedes apoyar en las tablas presentadas.

De asignación: los operadores de asignación transforman y asignan valores a variables en una sola operación. Por ejemplo: en la expresión "int x = 0;" se está asignando un valor de cero a la variable de tipo entero con nombre "x".

Operador	Operación realizada
=	Asignación sencilla
*=	Asignación de multiplicación
/=	Asignación de división
%=	Asignación de módulo
+=	Asignación de suma
-=	Asignación de resta

Aritméticos: Estos operadores nos permiten realizar cálculos con valores numéricos para obtener determinados resultados. Equivalen a las operaciones comunes que se utilizan en matemáticas. Por ejemplo: en la expresión "i + 25 * 10" se están sumando y multiplicando 3 valores distintos con ayuda de los operadores aritméticos.

Se pueden dividir en operadores aritméticos binarios y unarios. Los binarios trabajan con dos valores u operandos y son los siguientes:

Operador	Operación realizada
Suma (+)	Suma dos operandos
Sustracción (-)	Resta el segundo valor al primero
Multiplicación (*)	Multiplica dos operandos
División (/)	Divide un operando entre otro
Módulo (%)	Calcula el residuo de una división

Por otro lado, los operadores aritméticos unarios trabajan con un solo operando. Los que debes conocer son:

Operador	Descripción
Incremento (++)	Incrementa una unidad el valor entero de la variable
Decremento (- -)	Decrementa una unidad el valor entero de la variable

Lógicos: Se utilizan comúnmente en declaraciones condicionales, ayudan en la toma de decisiones determinando qué bloques de código deben ejecutarse en función del valor resultante según la tabla de verdades (el valor resultante siempre debe ser un booleano true o false). Equivalen a los procesos lógicos de disyunción (o), conjunción (y) y negación (no).

Operador	Descripción
&&	Verifica si todos los operandos son verdaderos, si es así entonces el resultado será verdadero
	Verifica si al menos uno de los operandos es verdadero, si es así entonces el resultado será verdadero
!	Niega al operando, es decir, devuelve el opuesto. Si es true, regresará como false

Comparación: Los operadores de comparación se utilizan para comparar dos valores o variables, proceso de suma importancia en la programación pues nos ayuda en la toma de decisiones y en estructuras condicionales. El valor de retorno de las expresiones que utilizan estos operadores es booleano, es decir, solo podrá ser verdadero o falso. Equivalen también a sus similares comúnmente utilizados en operaciones aritméticas y algebraicas.

Operador	Descripción
== (igual que)	True si los valores son iguales
!= (diferente de)	True si los valores son diferentes
> (mayor que)	True si el primer valor es mayor que el 2do
< (menor que)	True si el 1er valor es menor que el 2do
>= (mayor o igual que)	True si el 1er valor es mayor o igual al 2do
<= (menor o igual que)	True si el 1er valor es menor o igual al 2do

¡Nota! Es importante que observes con atención los operadores que utilizas al momento de construir una expresión, pues, debido a la similitud de algunos, es posible cometer pequeños errores de escritura y no lograr la solución correcta a tu problema.

- **Precedencia y construcción de expresiones**

La precedencia y el orden en que construyes expresiones en C afecta la agrupación y evaluación de los operandos, por ende es de suma importancia conocer qué tipo de operadores tienen prioridad.

Generalmente se utilizan signos de agrupación como paréntesis y corchetes para marcar jerarquías de evaluación dentro de una expresión.

La siguiente tabla resume la precedencia y asociatividad de operadores en C, se enumeran en orden de prioridad de mayor a menor. Los que aparecen en la misma línea o grupo tienen la misma prioridad.

Símbolos
[], (), .
&, *, +, -, !, ++, --
*, /, %
+, -
<, >, <=, >=
==, !=
&&
=, *=, /=, %=, +=, -=

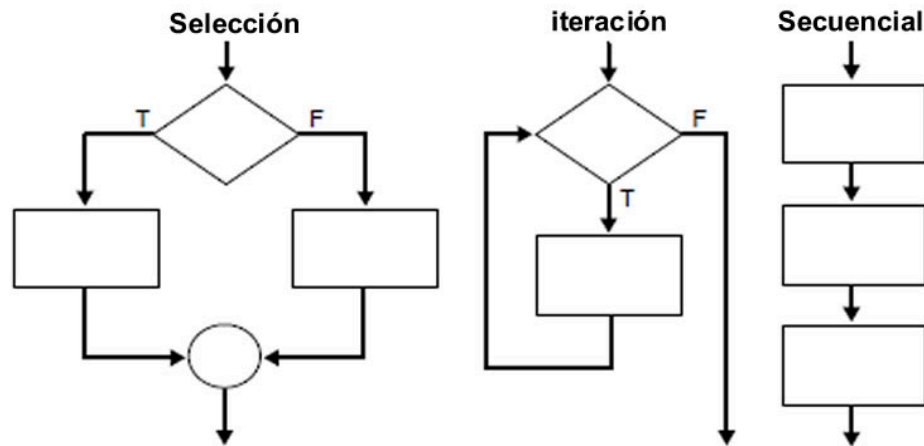
Para construir expresiones correctamente se debe tener siempre en cuenta la prioridad de evaluación de los operadores y operandos, además de la función específica de cada uno dentro de la expresión. De esta manera se pueden evaluar condiciones y crear operaciones que nos permitan construir funciones más complejas dentro de un programa.

3. Estructuras de control.

En programación, una estructura de control es el bloque de instrucciones encargado de controlar el flujo de ejecución de un programa. Son relevantes para trabajar en conjunto con operaciones básicas, pues realizan de manera eficiente tareas como evaluar condiciones o repetir ciclos de manera ordenada.

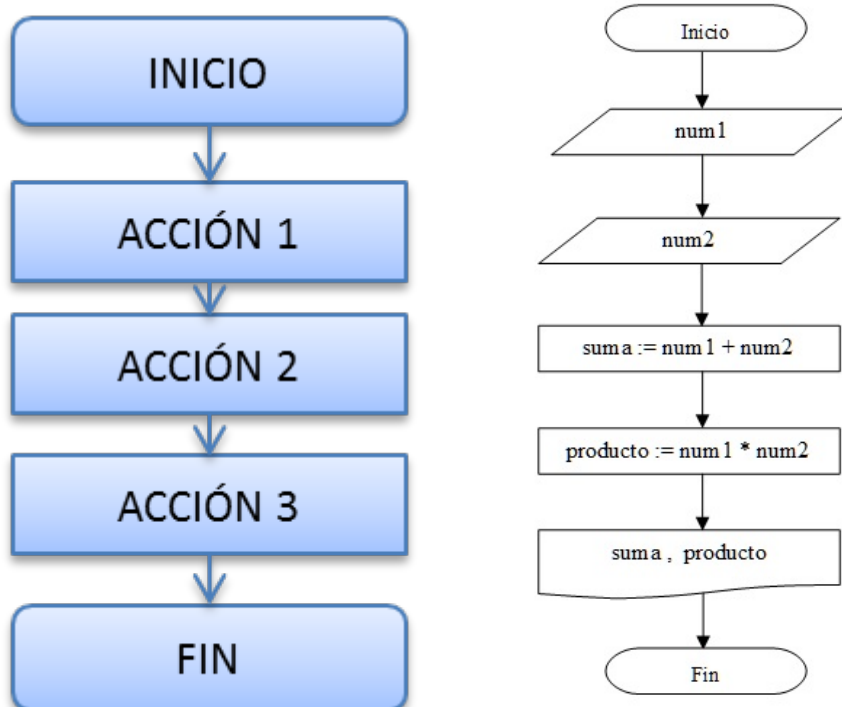
Dentro del lenguaje C podemos encontrar estructuras secuenciales, condicionales o de selección e iterativas. Cada una de ellas cumple con funciones distintas y son de utilidad en casos diferentes, conocer el flujo de trabajo que llevan a cabo es importante para construir expresiones correctas dentro de un código.

Siempre te puedes apoyar en diagramas de flujo o esquemas que te permitan entender mejor de qué manera se trabaja con cada estructura.



Repasando de manera superficial:

Estructuras secuenciales. Siendo las más básicas, ejecutan cada instrucción una después de otra, en el mismo orden en el que están escritas en el código.



Estructuras condicionales. Estas ejecutan un bloque de código u otro o, en algunos casos, saltan a una función o subprograma dentro del programa principal. Dependen siempre de que el valor de retorno de un condicional sea verdadero o falso. Si una condición se cumple, se ejecuta cierta acción, si no, se ejecuta otra.

En C podemos encontrar:

Condicional simple (if). Evalúa una condición y si es verdadera ejecuta un bloque de código. Si no se cumple solo se ignora.

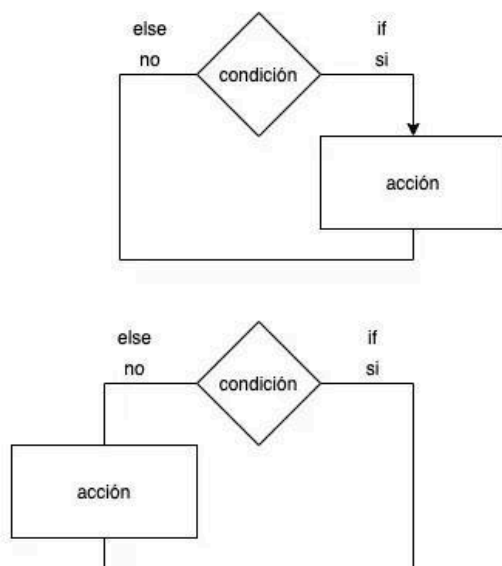
Condicional doble (if-else). Nos permite ejecutar un bloque de instrucciones en caso de que la condición se cumpla y otro diferente en caso de que sea falsa.

Condicional múltiple (if - else - if). Utilizado para evaluar varias condiciones de forma secuencial.

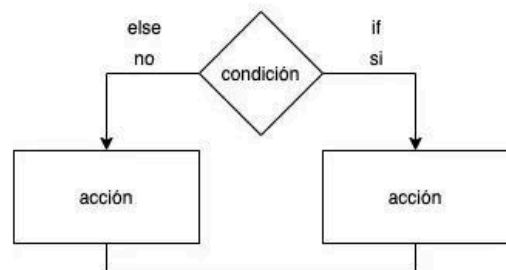
Selección múltiple (switch - case). Es una forma limpia y eficiente de trabajar múltiples condicionales dobles. Permite varios casos específicos que ejecutan diferentes expresiones según el valor de la condición.

Operador ternario. Una forma abreviada de escribir un if - else, solo ocupa una línea y es útil en instrucciones muy concretas.

ESTRUCTURA CONDICIONAL SIMPLE



ESTRUCTURA CONDICIONAL DOBLE

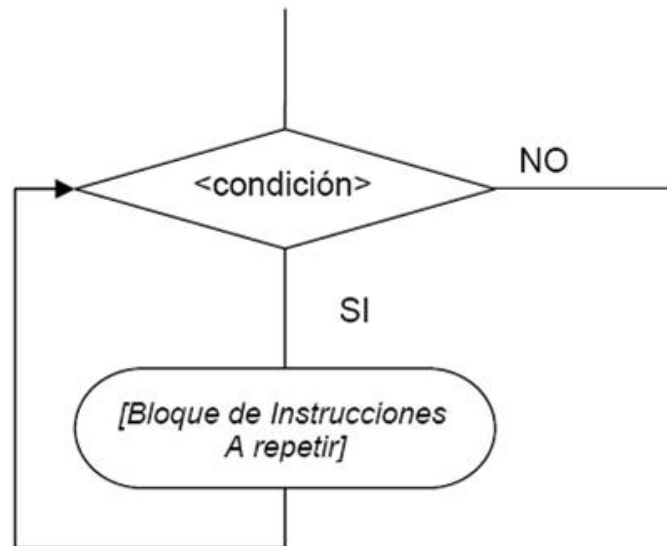


Estructuras iterativas. Las estructuras iterativas o de repetición permiten repetir un bloque de código tantas veces como se cumpla una condición. Dependen de que el valor de la condición siga siendo el mismo. Cuando se deja de cumplir, el código deja de ejecutarse. Las estructuras iterativas que se deben conocer esencialmente son el bucle for, el bucle while y el bucle do - while.

For. Útil cuando se conoce de antemano el número exacto de veces que se va a repetir un bloque de código. Agrupa en la misma línea la inicialización de una variable, la condición y el incremento o decremento de la misma.

While. Utilizado cuando el número de iteraciones no es fijo y depende de una condición a evaluar al principio de cada ciclo de repetición.

Do - while. Similar al while, con la diferencia de que garantiza que el bloque de código dentro se ejecutará mínimo una vez, puesto que la condición se evalúa al final de la estructura y no al principio como en el while.



4. Algoritmos y diagramas de flujo

El concepto de **algoritmo** dentro de computación es de suma importancia. Un algoritmo es un conjunto de reglas que determinan la secuencia de operaciones a seguir para resolver un problema específico. Debe cumplir con las siguientes características:

Finitud. Debe terminar después de cumplir con un número finito de pasos.

Definición. Cada paso debe estar definido con precisión, no debe haber ambigüedad.

Entrada. Se considera como entrada el conjunto de datos o información requerida para resolver el problema dado.

Salida. Es el conjunto de resultados obtenidos al aplicar el algoritmo.

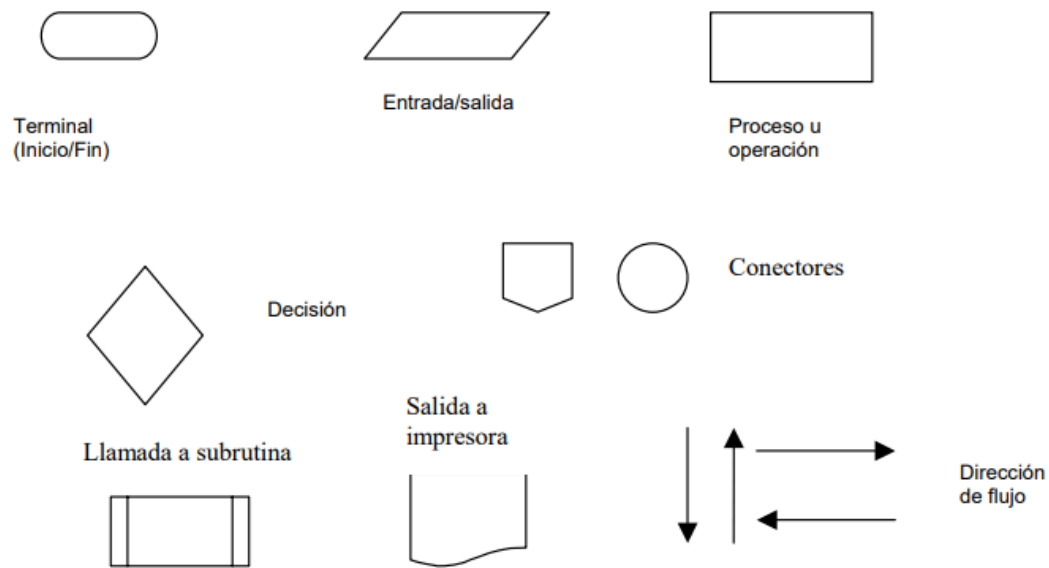
Efectividad. Debe llevar correcta y sencillamente a la solución del problema dado, debe ser simple y representable a través de esquemas (diagramas de flujo).

Por otro lado, los **diagramas de flujo** son la esquematización gráfica de un algoritmo.

La construcción correcta de los diagramas de flujo es de suma importancia, porque a partir de estos se escribe el código. Si el diagrama de flujo está bien realizado, el programa será fácil de escribir y corregir.

Los diagramas de flujo muestran con claridad cómo operan las estructuras de control utilizadas en la programación estructurada, constan de un conjunto de símbolos con diferentes significados y que se pueden conectar entre sí para crear expresiones completas.

La siguiente imagen ilustra los elementos básicos de los diagramas de flujo que debes conocer para crearlos correctamente:



Teniendo ya presente cómo funcionan todos los elementos vistos en este documento, puedes comenzar a construir tus propias soluciones a cientos de problemas. Recuerda siempre seguir un orden de trabajo basado en las diversas metodologías y técnicas de resolución de problemas que existen. Procura no omitir pasos, pues la base para lograr soluciones eficaces y bien hechas radica en trabajar con estos elementos.

Para complementar lo que aprendes en clase y lo visto en este documento puedes encontrar más información de utilidad en las siguientes páginas web, asimismo utilizadas como referencia para la creación de este material de apoyo:

<https://www.cs.buap.mx/~mtovar/doc/MP/Unidad1y2.pdf>

https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3065/mod_resource/content/1/UAPA-Solucion-Problemas-Algoritmos/index.html

<https://keepcoding.io/blog/resolucion-de-problemas-en-programacion/>

<https://www.universidadunie.com/blog/metodologias-de-programacion>

<https://www.lenovo.com/mx/es/glosario/variable>

<https://www.ibm.com/docs/es/tcamfma/6.3.0?topic=tesl-constants-variables-4>

<https://learn.microsoft.com/es-es/cpp/c-language/c-operators?view=msvc-170>

<https://profesores.dcb.unam.mx/users/alejandromra/Secuencial.pdf>